BSc (Hons) in Information Technology Specializing Data Science

Research Project - IT4010 **MLOPs Report**

Group ID: 25-26J-106

Project Title: Adaptive And Explainable Human-Robot Collaboration for Intelligent Sorting Systems

1. IT 22238962: Collaborative Texture Recognition
2. IT 22089618: Adaptive Pattern Recognition via Human Behavior
3. IT 22078032: Human-Led Selection Through Gaze-Guided Robotic Assistance
4. IT 22888716: Integrating Explainable AI and Adaptive Recommendations for Human Centered Robotic Sorting

		IT 22238962	IT 22089618	IT 22078032	IT 22888716
Data Pipeline	**Data Sources:**	RGB camera images captured via IP Webcam application; objects across 5 categories: plastic, glass, rubber, metal, and other collected under varied lighting, angles, backgrounds, and distances	Data is collected from classroom interactions using cameras and microphones, along with a labeled dataset of objects and patterns.	Custom dataset collected using RGB/CCTV cameras capturing facial images of participants looking at predefined gaze regions (upper-left, center, etc.). Includes variations in lighting, head pose, and facial features.	A custom dataset consisting of 2,080 images is used, capturing objects with different shapes (ball, box, pyramid) and colors (red, green, blue). Images include variations in lighting, object arrangement, overlap, and camera angles to simulate real-world educational environments.

	Data Preprocessing:	Image resizing to model input resolution, pixel normalization, noise reduction; augmentation via horizontal flipping, rotation, brightness adjustment, and zooming	Images are resized and augmented, while speech is converted to text and cleaned for better accuracy.	Image resizing, normalization, face-region extraction, noise reduction, and data augmentation (rotation, brightness changes, slight shifts) to improve model robustness and generalization.	Images are resized, normalized, and annotated using tools like Roboflow. Data augmentation techniques such as rotation and brightness adjustments are applied to improve robustness. Object labels are assigned based on color and shape combinations to support accurate detection and classification.
	Data storage and versioning	Dataset split into training and testing sets; images labeled by material category and stored locally for model training and evaluation	Data is stored in an organized way with version control to track changes and ensure consistency.	Dataset stored in structured folders by gaze class. Version control maintained manually (or using tools like Git/DVC if applicable) to track dataset updates and preprocessing changes.	Processed datasets are stored in structured formats for training and evaluation. Dataset versions are maintained to ensure reproducibility and consistency across experiments, especially when testing different machine learning models.

Model Development	**Model Selection**	Four models evaluated: Sequential CNN (0.44), EfficientNet (0.88), MobileNet (0.89), DINOv2 (0.98); DINOv2 selected as final model	A YOLO-based model is used for fast object detection, along with methods for pattern recognition.	ResNet-18 CNN architecture selected due to its balance between accuracy and computational efficiency compared to models like Mobile Net and VGG16.	**Object Detection Model** A YOLOv8 model is used for real-time object detection due to its high speed and accuracy. It identifies objects based on predefined classes (color + shape), enabling efficient perception in dynamic environments. **Recommendation Model** Machine learning models including Random Forest, XGBoost, and Logistic Regression are evaluated. The Random Forest model is selected due to its high accuracy and interpretability for determining optimal sorting strategies based on object distribution.

	Model Training	DINOv2 trained using self-supervised vision transformer approach; augmented dataset used to improve generalization and reduce overfitting	Models are trained using labeled data and transfer learning, with evaluation based on accuracy and speed.	Supervised learning using a labeled dataset. Training includes cross-entropy loss, backpropagation, validation monitoring, early stopping, and hyperparameter tuning (learning rate, batch size, epochs).	Models are trained using labeled datasets with an 80/20 train-test split. Techniques such as SMOTE are used to handle class imbalance. Performance is evaluated using accuracy and classification metrics to ensure reliability in real-time scenarios.
Model Integration	**Tools used**	Python-based deep learning framework; microcontroller for servo/sensor integration; IP Webcam for real-time image capture; ultrasonic sensor for object detection	Python, TensorFlow/PyTorch, and OpenCV are used to integrate vision, speech, and robotic components.	Python, OpenCV (for image processing), PyTorch/TensorFlow (for deep learning), Arduino IDE (for robotic control), and serial communication for integrating model output with robotic arm.	**Explainable AI (XAI)** SHAP (SHapley Additive Explanations) is integrated to provide interpretability. It explains how features such as object count and color influence the sorting decision, making the system transparent and suitable for educational use.

					System Integration Tools The system is developed using Python with libraries such as OpenCV for image processing and machine learning frameworks. Serial communication is used to connect the AI system with the Arduino-based robotic controller.
Model Deployment	**Testing Environments**	Controlled laboratory environment simulating real assistive educational settings; tested under continuous real-time operation	The system is tested in controlled environments simulating classroom conditions, including variations in lighting and object placement. This ensures reliability and real-time performance.	Controlled lab setup with RGB camera, predefined object workspace, and robotic arm. Tested under different lighting conditions and user variations.	The system is tested in controlled environments that simulate classroom conditions, including variations in object placement, lighting, and user interaction. This ensures stability and robustness in real-world educational settings.

	Deployment Platform	On-device (local hardware); microcontroller-based robotic system with integrated camera and sensors	The system runs on a computer or embedded device connected to cameras, sensors, and the robotic platform. The setup supports real-time interaction and is cost-effective for educational use.	Local machine (PC/laptop) connected to camera and robotic arm. Can be extended to embedded systems or edge devices in future.	The system runs on a computer connected to a camera (IP webcam) and an Arduino-controlled robotic arm. This setup ensures real-time processing and cost-effective deployment in educational institutions.
	Deployment Method	Real-time inference pipeline — image capture → DINOv2 classification → sensor validation → robotic actuation	The trained model is deployed as a real-time application that processes input data and controls the robotic system. It operates continuously to enable smooth and responsive interaction.	Real-time inference pipeline: capture image → preprocess → predict gaze → map to region → send command to robotic arm → execute pick-and-place action.	The trained models are deployed as a real-time pipeline that performs object detection, generates sorting recommendations, provides explanations, and controls the robotic arm for execution.
Future Enhancements	**Model Improvement:**	Edge deployment on resource-constrained devices; expansion to more material categories (wood, fabric, ceramics);	Future work includes improving accuracy, expanding datasets, and enhancing behavior recognition using advanced models.	Improve accuracy with larger and more diverse datasets, implement real-time optimization, reduce latency, explore lightweight models (e.g.,	Future work includes expanding the dataset, improving detection accuracy, and optimizing real-time performance. Additional object

		reinforcement learning for robotic control; integration of LiDAR, infrared depth, and hyperspectral sensors	Continuous learning can be added to adapt to new user behaviors over time.	Mobile Net), and integrate multimodal inputs (voice/gesture).	categories and more complex environments can be introduced. Enhancements include improving interactive explainability, adding voice-based explanations, and adapting the system to support personalized learning for differently-abled students.